Lab 6: Code and Branch Coverage Analysis Report

Name: Arvind Ajay Bharadwaj

SRN: PES1UG24AM052

Section: A

1. Application Overview

The application contains three functions:

- `reward_points()` — Calculates recycling reward points based on material, weight, and bonus eligibility.
- `collection_accepted()` — Determines whether a recycling collection is accepted based on weight, number of bags, and hazardous material status.
- `slot_status()` — Determines whether a pickup slot is open, late, or missed based on time remaining, morning/evening scheduling, and holidays.

The application contains 37 executable statements and 26 branch outcomes.

2. Code Coverage Analysis

2.1 Initial Results

The initial test suite contained only two test cases, covering paper recycling below the minimum weight and acceptance of a normal recycling collection.

Metric	Initial Result
Test cases	2
Passed	2
Executable statements	37
Missing statements	23
Application line coverage	38%

The initial coverage was low because several important execution paths, including reward calculations for higher weights, hazardous collections, and pickup slot statuses, were not tested.

2.2 Additional Test Cases

Six additional test cases were introduced to cover previously unexecuted statements.

These included:

1. Paper recycling above the minimum weight.
2. High-weight metal recycling with additional rewards.
3. Open morning pickup slots.
4. Hazardous collections within the permitted weight.
5. Evening holiday pickup statuses.
6. Rejection of overweight collections.

These tests exercised the main functional paths of all three application functions.

2.3 Final Code Coverage Results

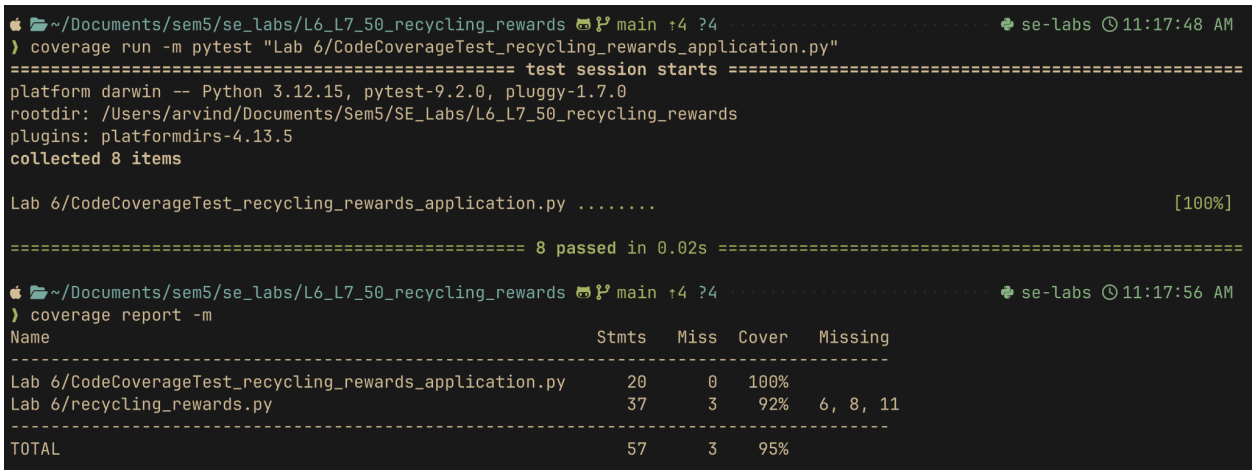
Metric	Result
Test cases	8
Passed	8
Executable statements	37
Covered statements	34
Missing statements	3
Application line coverage	**92%**

The code coverage increased from **38% to 92%**, an improvement of 54 percentage points.

The additional tests substantially improved execution coverage without requiring modifications to the application source code.

The final code coverage exceeded the minimum requirement of 90%.

2.4 Final Terminal Output



3. Branch Coverage Analysis

3.1 Initial Results

Branch coverage evaluates whether alternative outcomes of conditional statements have been executed, rather than only checking whether individual statements were reached.

The initial branch coverage report showed:

Metric	Initial Result
Executable statements	37
Branch outcomes	26
Missing statements	23
Missing branch outcomes	6
Combined coverage	32%

The existing tests did not adequately exercise alternative decisions involving invalid inputs, bonus eligibility, collection restrictions, and pickup scheduling.

3.2 Additional Branch Test Cases

Two additional test cases were introduced to cover exception-handling branches:

Test 1: Invalid Material

The `reward_points()` function was tested using an unsupported material (`"glass"`). The test verified that the function raised a `ValueError` with the expected message.

Test 2: Negative Weight

The same function was tested with a negative weight. The test verified that invalid weight input raised the appropriate `ValueError`.

These test cases exercised exception paths that had not been covered during the initial code coverage phase.

3.3 Final Branch Coverage Results

Metric	Result
Test cases	10
Passed	10
Executable statements	37
Covered statements	36
Total branch outcomes	26
Covered branch outcomes	24
Missing branch outcomes	2
Statement coverage	97.3%
Branch coverage	92.3%
Combined coverage reported	**95%**

The final results exceeded the 90% branch coverage requirement.

3.4 Analysis of Remaining Uncovered Paths

The final coverage report identified the following missing paths:

Line 11 — Bonus adjustment

The statement reducing the minimum required recycling weight for eligible non-metal materials was not executed.

This indicates that the tests did not exercise the specific combination of an eligible material with an active bonus.

Branch 21 → 23 — Non-metal high-weight reward

The false outcome of the metal-specific condition within the high-weight reward calculation remained untested.

This branch would be exercised by testing a non-metal material whose extra weight exceeds 20 kg.

These uncovered paths do not prevent the application from meeting the lab's required coverage threshold.

3.5 Final Terminal Output

```
~/Documents/sem5/se_labs/L6_L7_50_recycling_rewards main ↑5 ?5 ····· se-labs 11:25:10 AM
❯ coverage run --branch -m pytest "Lab 6/BranchCoverageTest_recycling_rewards_application.py"
============================================== test session starts ==============================================
platform darwin -- Python 3.12.15, pytest-9.2.0, pluggy-1.7.0
rootdir: /Users/arvind/Documents/Sem5/SE_Labs/L6_L7_50_recycling_rewards
plugins: platformdirs-4.13.5
collected 10 items

Lab 6/BranchCoverageTest_recycling_rewards_application.py ..........                                    [100%]

============================================== 10 passed in 0.02s ===============================================

~/Documents/sem5/se_labs/L6_L7_50_recycling_rewards main ↑5 ?5 ····· se-labs 11:25:18 AM
❯ coverage report -m
Name                                                                Stmts   Miss Branch BrPart  Cover   Missing
---------------------------------------------------------------------------------------------------------------
Lab 6/BranchCoverageTest_recycling_rewards_application.py              26      0      0      0   100%
Lab 6/recycling_rewards.py                                             37      1     26      2    95%   11, 21→23
---------------------------------------------------------------------------------------------------------------
TOTAL                                                                  63      1     26      2    97%
```

4. Comparison of Initial and Final Results

Metric	Initial	Final
Test cases	2	10
Application statement coverage	38%	97.3%
Application branch coverage	20/26 outcomes*	24/26 outcomes
Branch-enabled combined coverage	32%	95%
Tests failed	0	0

The initial report recorded 6 missing branch outcomes out of 26. However, substantial statement coverage was also missing, so the combined percentage was considerably lower.

All added tests passed successfully.

5. Observations

1. The initial test suite successfully tested basic application functionality but left many statements and conditional outcomes uncovered.
2. Adding tests for normal, boundary, exceptional, and alternative execution cases improved coverage substantially.
3. Code coverage alone was insufficient to determine whether all decisions were tested. Branch coverage provided additional information about untested conditional outcomes.
4. Exception-handling tests were particularly useful because they exercised paths not reached by valid inputs.
5. High coverage improves confidence in the tests, but it does not guarantee that the application is free from logical defects.
6. The final test suite achieved 97.3% statement coverage and 92.3% branch coverage, satisfying both requirements.

6. Conclusion

The Code and Branch Coverage Analysis experiment was successfully completed using pytest and coverage.py.

The application initially had 38% statement coverage and 32% combined branch-enabled coverage. By designing additional test cases targeting unexecuted statements, decision branches, and invalid inputs, the final results improved to **97.3% statement coverage, 92.3% branch coverage, and 95% combined coverage**.

All 10 test cases passed successfully.

The experiment demonstrated the importance of systematic test-case design and the complementary roles of statement and branch coverage in software testing. The required minimum coverage of 90% was achieved for both measures without modifying the application implementation.